# 🎓 جزوه طلایی سیستم‌عامل

## فصل ۱۰: حافظه مجازی - بخش هفتم (مدیریت کوبیدگی و فشرده‌سازی)

### ۱. چرخه مرگبار کوبیدگی (The Vicious Cycle of Thrashing)

چرا کوبیدگی اتفاق می‌افتد و چرا سیستم‌عامل خودش باعث بدتر شدن آن می‌شود؟

1. **شروع ماجرا:** پردازه‌ها به صفحات بیشتری نیاز دارند ⬅ خطای صفحه (Page Fault) زیاد می‌شود.
2. **انتظار برای دیسک:** پردازه‌ها در صف دستگاه صفحه‌بندی (Paging Device) منتظر می‌مانند.
3. **بیکاری CPU:** چون همه منتظر دیسک هستند، صف آماده (Ready Queue) خالی می‌شود و بهره‌وری CPU (Utilization) کاهش می‌یابد.
4. **اشتباه استراتژیک سیستم‌عامل:** زمان‌بند CPU می‌بیند بهره‌وری پایین است، فکر می‌کند سیستم بیکار است! پس **درجه چندبرنامگی (Degree of Multiprogramming) را افزایش می‌دهد** (پردازه جدید وارد می‌کند).
5. **فاجعه:** پردازه جدید هم رم می‌خواهد ⬅ قاب از بقیه می‌گیرد ⬅ خطای صفحه بیشتر می‌شود ⬅ CPU بیکارتر می‌شود ⬅ سیستم‌عامل باز هم پردازه اضافه می‌کند!
    - **نتیجه:** نمودار بهره‌وری CPU ناگهان سقوط می‌کند.

### ۲. مدل همسایگی (Locality Model)

برای جلوگیری از کوبیدگی، باید رفتار برنامه را درک کنیم.

- **اصل همسایگی:** یک برنامه به طور تصادفی به حافظه دسترسی ندارد، بلکه از یک "همسایگی" به همسایگی دیگر می‌رود.
- **تعریف:** همسایگی مجموعه‌ای از صفحات است که فعالانه با هم استفاده می‌شوند (مثلاً صفحات مربوط به یک حلقه for یا یک تابع خاص).
- **شرط جلوگیری از کوبیدگی:** باید به پردازه **تعداد قاب کافی برای پوشش همسایگی فعلی‌اش** بدهیم. اگر کمتر بدهیم، مدام خطای صفحه می‌دهد.

## ۳. مدل مجموعه کاری (Working Set Model)

این مدل سعی می‌کند "همسایگی" را به صورت ریاضی تقریب بزند.

- **پنجره مجموعه کاری ($\Delta$):** یک عدد ثابت (مثلاً ۱۰,۰۰۰ دستورالعمل اخیر).
- **مجموعه کاری ($WSS_i$):** مجموعه‌ای از صفحات منحصربه‌فردی که پردازه $P_i$ در بازه $\Delta$ اخیر به آن‌ها مراجعه کرده است.
  - اگر صفحه استفاده شود ⬅ در مجموعه می‌ماند.
  - اگر استفاده نشود ⬅ بعد از گذشت زمان $\Delta$ از مجموعه خارج می‌شود.

**فرمول حیاتی:**

$$D = \Sigma WSS_i$$

- $D$: کل تقاضای فریم‌ها (Total Demand).
- $m$: کل فریم‌های موجود در RAM.

**استراتژی سیستم‌عامل:**

1. اگر $D > m$ باشد (تقاضا بیشتر از موجودی) ⬅ **کوبیدگی رخ می‌دهد.**
2. **راهکار:** یکی از پردازه‌ها را انتخاب و **کلاً معلق (Suspend)** کن (صفحاتش را خالی کن) تا $D$ کم شود و بقیه بتوانند راحت نفس بکشند!

## ۴. فرکانس خطای صفحه (Page-Fault Frequency - PFF)

یک روش مستقیم‌تر و عملی‌تر برای کنترل کوبیدگی. به جای محاسبه پیچیده مجموعه کاری، نرخ خطا را چک می‌کنیم.

- **منطق:**
  - اگر نرخ خطای صفحه **خیلی بالا** باشد ⬅ پردازه قاب کم دارد ⬅ **به آن قاب بده.**
  - اگر نرخ خطای صفحه **خیلی پایین** باشد ⬅ پردازه قاب زیادی دارد (انحصارگر است) ⬅ **از آن قاب بگیر.**
- **مکانیزم:** دو حد بالا (Upper Bound) و پایین (Lower Bound) تعریف می‌کنیم و تعداد قاب‌ها را بین این دو تنظیم می‌کنیم تا سیستم متعادل بماند.

### ۵. فشرده‌سازی حافظه (Memory Compression)

یک تکنیک مدرن (مخصوصاً در موبایل‌ها و macOS) که جایگزین مبادله (Swap) سنتی شده است.

- **ایده:** به جای اینکه صفحه قربانی را در دیسک بنویسیم (که کند است)، آن را **فشرده (Zip)** می‌کنیم و در یک بخش کوچک از خودِ RAM نگه می‌داریم.
- **مزیت:**
  - کاهش I/O دیسک (سرعت بالاتر).
  - استفاده بهینه‌تر از فضای RAM.
- **مثال:** به جای داشتن ۳ صفحه معمولی، آن‌ها را فشرده کرده و در ۱ قاب جا می‌دهیم (۲ قاب آزاد می‌شود).

### 📝 خلاصه نکات طلایی بخش هفتم:

1. **علت اصلی کوبیدگی:** افزایش کورکورانه درجه چندبرنامگی توسط سیستم‌عامل وقتی I/O زیاد است.
2. **شرط کوبیدگی:** وقتی مجموع اندازه مجموعه‌های کاری ($D$) از کل رم ($m$) بیشتر شود.
3. **WSS (Working Set):** تقریب همسایگی بر اساس $\Delta$ دستور اخیر.
4. **PFF:** تنظیم تعداد قاب‌ها بر اساس نرخ خطا (بالا=بده، پایین=بگیر).
5. **Memory Compression:** فشرده‌سازی در رم به جای نوشتن در دیسک (افزایش سرعت).